

Ejercicio 1

Dada la siguiente definición recursiva de la función f (que toma como entrada 3 números enteros positivos y devuelve una cadena):

$$f(a, b, c) = \begin{cases} \text{cadena}(a + b + c) \cup "y", & a < 3 \vee b < 3 \vee c < 3 \\ \text{cadena}(a * b * c) \cup "z", & a < 5 \vee (b < a \wedge b < c) \vee c < 5 \\ f(a - 1, \frac{b}{2}, c - 3) \cup "+", & a \text{ es par} \wedge b \text{ es par} \wedge c \text{ es par} \\ f(a - 4, \frac{b}{3}, c - 4) \cup "\#", & \text{en otro caso} \end{cases}$$

siendo $\cup$ un operador que representa la concatenación de cadenas, y $\text{cadena}(i)$ un método que devuelve una cadena a partir de un entero i .

SE PIDE:

Resuelva cada apartado en una carilla diferente.

- Proporcione una solución eficiente recursiva no final.
- Proporcione una solución eficiente recursiva final.
- Proporcione una solución eficiente en notación funcional.
- Proporcione una solución eficiente iterativa usando *while*.
- Calcule el orden de complejidad del algoritmo iterativo que ha desarrollado en el apartado d), definiendo previamente su tamaño n y la función $T(n)$. Suponga que el orden de complejidad de la concatenación de cadenas es constante.

Tiempo estimado: 60 min.

En las soluciones se usan estas soluciones auxiliares. El uso de funciones auxiliares es opcional para resolver este ejercicio.

```
/*
 * Funciones Auxiliares
 */

public static Boolean esCasoBase(Integer a, Integer b, Integer c) {
    Boolean caso1 = a < 3 || b < 3 || c < 3;
    Boolean caso2 = a < 5 || (b < a && b < c) || c < 5;
    return caso1 || caso2;
}

public static String solucionBase(Integer a, Integer b, Integer c) {
    String r = "";
    if (a < 3 || b < 3 || c < 3) {
        r = (a + b + c) + "y";
    } else if (a < 5 || (b < a && b < c) || c < 5) {
        r = (a * b * c) + "z";
    }
    return r;
}
```

a) Solución recursiva no final

```
public static String f_no_final(Integer a, Integer b, Integer c) {
    if (a < 3 || b < 3 || c < 3) {
        return (a + b + c) + "y";
    } else if (a < 5 || (b < a && b < c) || c < 5) {
        return (a * b * c) + "z";
    } else if (a % 2 == 0 && b % 2 == 0 && c % 2 == 0) {
        return f_no_final(a - 1, b / 2, c - 3) + "+";
    } else {
        return f_no_final(a - 4, b / 3, c - 4) + "#";
    }
}
```

b) Solución recursiva final

```
public static String f_final(Integer a, Integer b, Integer c) {  
    return f_final(a,b,c,"");  
}  
  
public static String f_final(Integer a, Integer b, Integer c, String acum) {  
    if (a < 3 || b < 3 || c < 3) {  
        return (a + b + c) + "y" + acum;  
    } else if (a < 5 || (b < a && b < c) || c < 5) {  
        return (a * b * c) + "z" + acum;  
    } else if (a % 2 == 0 && b % 2 == 0 && c % 2 == 0) {  
        return f_final(a - 1, b / 2, c - 3, "+" + acum);  
    } else {  
        return f_final(a - 4, b / 3, c - 4, "#" + acum);  
    }  
}
```

c) Solución en notación funcional

```
/*  
 * Solución funcional  
 */  
  
public static String f_funcional(Integer a, Integer b, Integer c) {  
    ejercicio res = Stream.iterate(ejercicio.of(a, b, c, ""), t -> t.siguiente())  
        .filter(p -> p.esCasoBase()).findFirst().get();  
    return solucionBase(res.a(), res.b(), res.c()) + res.ac();  
}  
  
public static record ejercicio(Integer a, Integer b, Integer c, String ac) {  
    public static ejercicio of(Integer a, Integer b, Integer c, String ac) {  
        return new ejercicio(a, b, c, ac);  
    }  
  
    public Boolean esCasoBase() {  
        return Ejercicio.esCasoBase(a, b, c);  
    }  
  
    public ejercicio siguiente() {  
        Integer a1 = a(), b1 = b(), c1 = c();  
        String ac1 = ac();  
        if (a1 % 2 == 0 && b1 % 2 == 0 && c1 % 2 == 0) {  
            ac1 = String.format("%s", "+") + ac1;  
            a1 = a1 - 1;  
            b1 = b1 / 2;  
            c1 = c1 - 3;  
        } else {  
            ac1 = String.format("%s", "#") + ac1;  
            a1 = a1 - 4;  
            b1 = b1 / 3;  
            c1 = c1 - 4;  
        }  
        return ejercicio.of(a1, b1, c1, ac1);  
    }  
}
```

d) Solución iterativa con while

```

/*
 * Iterativa usando while
 */
public static String f_while(Integer a, Integer b, Integer c) {
    Integer a_w = a;
    Integer b_w = b;
    Integer c_w = c;
    String acum = "";
    while (!esCasoBase(a_w, b_w, c_w)) {
        if (a_w % 2 == 0 && b_w % 2 == 0 && c_w % 2 == 0) {
            acum = "+" + acum;
            a_w = a_w - 1;
            b_w = b_w / 2;
            c_w = c_w - 3;
        } else {
            acum = "#" + acum;
            a_w = a_w - 4;
            b_w = b_w / 3;
            c_w = c_w - 4;
        }
    }
    acum = solucionBase(a_w, b_w, c_w) + acum;
    return acum;
}

```

e) Calcule el orden de complejidad del algoritmo iterativo, definiendo previamente su tamaño n y la función T(n)

- Tamaño - n = b

En el caso peor, b siempre es mayor que a o c (a o c se pueden hacer negativos pero b nunca lo será). El tamaño puede tomarse como el valor de b.

Por otro lado, si alguna vez a, b y c son pares, a partir de la siguiente iteración o llamada, a y c ya nunca no serán, por lo que la recurrencia se dará dividiendo b por 3.

- La forma recursiva es:

$$T(n) = 1 * T(n/3) + 1$$

- Resolvemos $T(n) = T(n/3)$

$$[\text{Caso B: } a=1, b=3, d=0, p=0] [a > b^d = 1=3^0]$$

$$\theta(T(n)) = n^d * \log^{p+1} n = n^0 * \log^{p+1} n$$

- De los dos apartados anteriores concluimos:

$$T(n) = \log(n)$$